

AN INTERNSHIP PROJECT REPORT ON :-

# **“Research on Open-Source LLMs for AI-Augmented Vulnerability Assessment”**

*Submitted in partial fulfillment of the requirement for the award  
of a certificate of internship programme in :-*

**JIO PLATFORMS LIMITED**

**Submitted By: -**

Vanshika Mohiley

**Under the supervision of: -**

Mr. Gaurav Kudaisya

## TABLE OF CONTENTS

|                                              | <b>Page</b> |
|----------------------------------------------|-------------|
| Table of Content                             | i           |
| Declaration                                  | ii          |
| Acknowledgement                              | iii         |
| List of Tables                               | iv          |
| List of Symbols and Abbreviations            | v           |
| <b>Chapter</b>                               |             |
| 1     Abstract                               | 1           |
| 2     Introduction                           | 2           |
| 3     Background                             | 6           |
| 4     Comparative LLM Study-Methodology      | 12          |
| 5     Comparative Results & Pros/cons Matrix | 20          |
| 6     Prototype Architecture                 | 34          |
| 7     Validation Results & Failure Analysis  | 39          |
| 8     Conclusion & Future Work               | 45          |
| References                                   | 50          |

## **CANDIDATE’S DECLARATION**

I, Vanshika Mohiley hereby declare that the report entitled “Research on Open-Source LLMs for AI-Augmented Vulnerability Assessment” which is being submitted to Project Mentor in **Jio Platforms Limited** is my authentic work carried out during internship.

I declare that my work has not been submitted in part or in full to any other organisation for the award of any certificate.

Vanshika Mohiley

## **ACKNOWLEDGEMENTS**

While bringing out this project to its final form, I came across people whose contributions helped me in various ways in my research and it is my proud privilege to express the feeling of gratitude to them. Firstly I would like to thank the organisation Jio Platform Limited for giving me this opportunity to pursue this internship with them and helping me each step of the way. I express my deep sense of gratitude to my mentor Mr. Gaurav Kudaisya for his complete support and encouragement from the start of the internship. I am proud to record that I had the opportunity to work under his supervision and in this organisation.

Vanshika Mohiley

# LIST OF TABLES

## List of Tables

| Table No. | Table Title                                                      |
|-----------|------------------------------------------------------------------|
| Table 1   | Comparison of Open-Source LLMs Evaluated                         |
| Table 2   | Dataset Composition and Task Distribution                        |
| Table 3   | Aggregate Model Performance Results                              |
| Table 4   | Novel CVE Detection Results                                      |
| Table 5   | RAG vs. No-RAG Performance Comparison                            |
| Table 6   | Prototype Validation Results — Base, RAG and Full Configurations |
| Table 7   | Accuracy Improvement Delta Across Configurations                 |
| Table 8   | Prototype Validation Confusion Results                           |
| Table 9   | Failure Mode Taxonomy                                            |
| Table 10  | Summary of Validation Findings                                   |

## **LIST OF SYMBOLS AND ABBREVIATIONS**

- API: Application Programming Interface
- CERT-In: Indian Computer Emergency Response Team
- CII: Critical Information Infrastructure
- CIS: Center for Internet Security
- CPE: Common Platform Enumeration
- CVE: Common Vulnerabilities and Exposures
- CVSS: Common Vulnerability Scoring System
- DPDP Act: Digital Personal Data Protection Act
- eBPF: Extended Berkeley Packet Filter
- GGUF: GPT-Generated Unified Format
- IPsec ESP: IP Security Encapsulating Security Payload
- IT Act: Information Technology Act
- MoE: Mixture of Experts
- NCIIPC: National Critical Information Infrastructure Protection Centre
- NVD: National Vulnerability Database
- OpEx: Operational Expenditure
- QLoRA: Quantized Low-Rank Adaptation
- RBAC: Role-Based Access Control
- RHEL: Red Hat Enterprise Linux
- RHSA: Red Hat Security Advisory
- RLVR: Reinforcement Learning from Verifiable Rewards
- SFT: Supervised Fine-Tuning
- SIEM: Security Information and Event Management
- SUID: Set User ID
- SWA: Sliding Window Attention
- TRAI: Telecom Regulatory Authority of India
- UAF: Use-After-Free
- USN: Ubuntu Security Notice
- VRAM: Video Random-Access Memory
- YAML: YAML Ain't Markup Language

## ABSTRACT

This report presents research conducted using open-source, on-premise Large Language Models (LLMs) for AI-augmented vulnerability assessment in Telecom Service Provider (TSP) environments. Frontier cloud-hosted models (Claude, GPT-4, Gemini) offer strong security-reasoning capability, but transmitting infrastructure scan data to third-party APIs conflicts with India's DPDP Act 2023, NCIIPC Critical Information Infrastructure guidelines, and TRAI's telecom security regulations — making cloud LLMs legally and operationally unusable for scanning TSP production systems. This creates the central research question: which open-source LLM, deployed entirely on-premise, can deliver adequate accuracy for this task? The research was conducted in three stages. First, six open-source LLMs (DeepSeek-Coder V2 16B, Gemma 2 9B, Llama 3.1 8B, Mistral 7B, Phi-3.5 Mini, Qwen2.5-Coder 7B) were comparatively evaluated on a 100-item benchmark spanning CVE accuracy, version reasoning, configuration-misconfiguration detection, remediation specificity, hallucination rate, structured-output reliability, and handling of novel (post-training-cutoff) CVEs. Second, results from this evaluation informed selection of a suitable model configuration. Third, a working prototype was built integrating local host-artifact collection, local CVE enrichment, retrieval-augmented generation (RAG), local LLM inference via Ollama, structured JSON validation, and reporting — with all processing kept inside the network boundary.

Results showed no single model was universally optimal; performance depended heavily on architecture, task, and whether RAG augmentation was used. Qwen2.5:7B with mandatory RAG emerged as the preferred configuration. In 30-case validation, the full prototype pipeline achieved 100% precision, recall, F1, JSON validity, and CVSS-tier accuracy — compared to 0% for a bare base model and ~45–61% for RAG alone — demonstrating that combining deterministic evidence collection, local retrieval, explicit version-reasoning constraints, and output validation is essential for reliable, sovereignty-compliant AI-assisted vulnerability assessment.

## Chapter 2 :- INTRODUCTION

### 2.1 Problem Statement

Large language models have reached a level of capability where they can meaningfully assist with security-relevant reasoning like interpreting CVE descriptions, correlating installed software versions against known vulnerabilities, reviewing configuration files against hardening baselines, and generating remediation guidance.

Frontier cloud-hosted models such as **Claude(Anthropic), GPT-4(openAI), and Gemini(Google)** demonstrate strong performance on exactly these tasks, and could, in principle, significantly augment vulnerability assessment workflows for network operators.

However, for a **Telecom Service Provider (TSP)** operating under Indian jurisdiction, using cloud-based frontier LLMs for security scanning of production infrastructure is not just inadvisable — it is legally and operationally untenable as vulnerability scan of a TSP's infrastructure produces artefacts that are inherently sensitive like:-

- Installed package lists with version strings (which can reveal network topology)
- Kernel parameters and configuration files (which expose hardening posture)
- Running service names and port listings (which disclose deployed capabilities)
- Container image identifiers (which reveal vendor and software stack).

Transmitting any of these artefacts to a third-party cloud API — regardless of which provider hosts it — means that sensitive operational data about **Critical Information Infrastructure (CII)** leaves the sovereign boundary of the operator's network.

This directly conflicts with India's ***Digital Personal Data Protection Act, 2023 (DPDP Act)***, which imposes data localisation and processing obligations on data fiduciaries, and with ***NCIIPC guidelines*** governing the protection of Critical Information Infrastructure, of which telecom is a designated sector. It further sits in tension with ***TRAI's Telecommunications (Security of Licensed Telecom Networks) Regulations, 2022***, which require operators to restrict access to network configuration data to authorised personnel and systems operating within defined boundaries.

Taken together, these constraints mean that no matter how capable a frontier cloud LLM is at vulnerability reasoning, it cannot be lawfully or safely used to process a TSP's live infrastructure data.

This creates a practical dilemma:- **”The operational value of AI-augmented vulnerability assessment is real, but the only LLMs currently at frontier capability are cloud-hosted and off-limits. The alternative is open-source LLMs with publicly available weights, deployed entirely on-premise which offer a path that satisfies the legal and operational constraints, provided such a model can deliver adequate accuracy for the task.”**

Whether that is actually achievable, and which open-source model performs best, has not been systematically established. This is the gap this research addresses.



## 2.2 Research Questions

### Core Research Question:-

*Given that frontier cloud LLMs are legally and operationally off-limits for security scanning in a TSP context, which open-source LLM — deployable fully on-premise — provides the best capability for AI-augmented vulnerability assessment?*

This decomposes into:

1. **RQ1 (Landscape):-** What open-source LLMs are viable candidates for on-premise deployment in a TSP environment, given constraints on hardware, licensing, and demonstrated capability on technical/security-adjacent tasks?
2. **RQ2 (Comparative Capability):** Across a defined set of security-assessment dimensions — CVE accuracy, version-specific vulnerability reasoning, configuration misconfiguration detection, remediation quality, hallucination rate, structured output reliability, and handling of novel (post-training-cutoff) CVEs , how do these candidate models compare, and what are the specific trade-offs of each?
3. **RQ3 (Feasibility):** Can a working prototype be built that uses the best-identified open-source model to deliver a functional, accurate, AI-augmented vulnerability scanner, while guaranteeing that no scan data, CVE context, or model inference leaves the operator's network boundary?

## **2.3 Scope and Contributions**

This research is in *scope* of the following parameters:-

- Linux-based operating systems (RHEL 8/9, CentOS Stream, Ubuntu 22.04/24.04, Debian 12)
- Container runtimes and orchestration (Docker, containerd, CRI-O, runc, podman, Kubernetes)
- Windows Server 2019/2022(wmic/Get-WmiObject,Get-service,netstat,Get-HotFix)
- CVE vulnerability data drawn from NIST NVD, Red Hat Security Advisories, Ubuntu Security Notices, the Debian Security Tracker, and GitHub Security Advisories.
- Six open-source LLMs evaluated as primary candidates — **Llama 3.1 8B, Mistral 7B Instruct, Qwen2.5-Coder 7B, DeepSeek-Coder V2 Lite, Phi-3.5 Mini, and Gemma 2** - selected for their open licensing, feasibility of on-premise deployment on modest hardware, and demonstrated capability on technical text tasks.

The *contributions* of this work are threefold:

1. A reproducible, evidence-based comparative evaluation of six open-source LLMs across seven security-assessment dimensions, using a purpose-built 100-item benchmark dataset with verified ground truth.
2. A structured pros-and-cons analysis identifying the specific strengths, weaknesses, and failure modes of each candidate model for vulnerability assessment use cases — including empirically measured hallucination rates and structured-output reliability.
3. A working, locally-deployable prototype vulnerability scanner that operationalises the best-performing model, demonstrably enforcing a zero-external-network-call architecture, thereby satisfying DPDP Act, NCIIPC, and TRAI compliance requirements for CII operators.

## **Chapter 3:- BACKGROUND**

### **3.1 Frontier LLMs for Security — Capability and Constraint**

Frontier language models have demonstrated measurable capability on security-relevant tasks. For example :-

- Given a CVE identifier, a frontier model can explain the underlying vulnerability, identify affected components, assess exploitability under a described environment, and suggest version-specific remediation, synthesising information across multiple sources in seconds.
- Given a configuration file ( sshd\_config, Kubernetes pod spec), these models can identify deviations from recognised hardening baselines like the CIS Benchmarks or 3GPP TS 33.117, and explain the associated risk.
- They can also reason across multiple artefacts simultaneously — correlating an installed package version with a CVE and a service's network exposure to determine whether a vulnerability is actually exploitable on a specific host — and can reliably produce structured, schema-conformant output suitable for downstream automation such as SIEM ingestion or ticketing.

These are exactly the capabilities a vulnerability scanner needs. The obstacle is not capability; it is where the model runs and what happens to the data sent to it.

#### **3.1.1 Why TSPs Cannot Use Frontier Cloud Models**

A security scan of a TSP's infrastructure produces artefacts that are sensitive by nature such as installed package lists with version strings, kernel parameters and configuration files, running service names and port listings and container image identifiers. Sending any of this to a cloud API — irrespective of provider or contractual assurances — moves sensitive operational data about Critical Information Infrastructure outside the operator's sovereign boundary.

Four distinct legal, operational and one further, non-legal constraint make this untenable :-

| <b>Constraint</b>                        | <b>Explanation</b>                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DPDP Act 2023 — Data Localisation</b> | The Digital Personal Data Protection Act, 2023 imposes obligations on data fiduciaries regarding the processing of personal data. Scan artefacts from TSP infrastructure may contain subscriber-related operational data. Processing this via a foreign cloud API risks violating Section 16 of the Act (transfer of personal data outside India) and the Act's broader data-localisation intent. |
| <b>NCIIPC CII Guidelines</b>             | TSPs are designated Critical Information Infrastructure operators under the IT Act 2000 and NCIIPC guidelines. Transmitting infrastructure configuration data ,even operational metadata to an external cloud service is inconsistent with CII protection requirements around information sovereignty.                                                                                            |
| <b>TRAI Security Framework</b>           | The Telecommunications (Security of Licensed Telecom Networks) Regulations, 2022 require operators to maintain the security of network configuration data and restrict its access to authorised personnel and systems operating within defined boundaries.                                                                                                                                        |
| <b>Operational Security</b>              | Configuration artefacts from a TSP's core network (NF package versions, sysctl settings, container image identifiers) reveal network topology, deployed capabilities, and hardening posture. Exposing this to a third-party API, even under NDA, introduces a supply-chain risk inconsistent with telecom security practice.                                                                      |
| <b>Availability</b>                      | A vulnerability scanner for production infrastructure needs to be usable offline, deterministically, and auditably. A cloud API dependency introduces availability risk (an internet outage takes the scanner offline) and removes the operator's ability to audit precisely what data was processed and when.                                                                                    |

### 3.1.2 Relevant Legal Provisions

Given below are the specific statutory and regulatory provisions underpinning the on-premise requirement of vulnerability assessment:-

#### 1. DPDP Act 2023 (India)

- **Section 16** - Processing of personal data outside India.
  - i) The Central Government may, by notification, restrict the transfer of personal data by a Data Fiduciary for processing to such country or territory outside India as may be so notified.
  - ii) Nothing contained in this section shall restrict the applicability of any law for the time being in force in India that provides for a higher degree of protection for or restriction on transfer of personal data by a Data Fiduciary outside India in relation to any personal data or Data Fiduciary or class thereof.
- **Section 8** – Indicates the Obligations of Data Fiduciaries (lawful processing, purpose limitation). A **Data Fiduciary** is any person or company that decides why and how personal data is processed.
- **Section 10** - Outlines the extra, stricter rules for organizations called significant data fiduciaries labelled based on specific risk factors. TSPs are likely to be designated as such, carrying additional obligations including data localisation.

#### 2. NCIIPC Guidelines

- The National Critical Information Infrastructure Protection Centre designates CII sectors, including telecom, and requires that CII operators not expose infrastructure configuration data to entities outside their control boundary.

#### 3. TRAI Regulations 2022

- The Telecommunications (Security of Licensed Telecom Networks) Regulations require licensed operators to implement security controls restricting access to network configuration data — sending such data to a cloud API runs contrary to the spirit of these controls.

#### 4. IT Act 2000-

- **Section 70B** – CERT-In was designated as national agency for Incident response. 2022 directions require all service providers to report incidents and maintain logs within India. Running a vulnerability scanner which sends data to external systems may conflict with log sovereignty requirements.

### 3.2 Locally-Deployed Open-Source LLMs

Open-source LLMs with publicly available weights can be downloaded once, run entirely on-premise, and never transmit data externally. This architecture directly satisfies the legal and operational constraints described in Section 3.1 and can replace Cloud Frontier LLMs but they are not specifically build for such tasks.

| Property           | Cloud Frontier LLM                                         | Open-Source On-Premise LLM                                               |
|--------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| Data residency     | Data leaves operator boundary - DPDP Act risk              | Compliant as all data remains on-premise.                                |
| DPDP Act 2023      | Likely violation for CII scan data                         | Compliant - no external processing                                       |
| Availability       | Requires internet; API outages affect scanning             | Fully offline; no external dependency                                    |
| Auditability       | Black-box; cannot audit what was processed                 | Full auditability of model weights and inference                         |
| Cost at scale      | Per-token API cost × thousands of hosts = significant OpEx | One-time infrastructure cost; zero per-scan cost                         |
| Security reasoning | State-of-the-art; continuously updated                     | Depends on model and RAG pipeline; can be competitive with proper design |

|               |                                                  |                                                                       |
|---------------|--------------------------------------------------|-----------------------------------------------------------------------|
| CVE currency  | Training cutoff + live search (in some products) | Requires RAG with a live NVD feed for post-cutoff CVEs                |
| Customisation | Cannot fine-tune; limited by API                 | Can be fine-tuned on security-specific data for domain specialisation |

### **Trade-off:**

- Open-source on-premise deployment resolves every legal and operational blocker, but at the cost of raw model capability compared to frontier systems.
- **Whether that capability gap is small enough to be closed with careful architecture — retrieval-augmented generation (RAG) against a live NVD feed, prompt engineering, and possibly fine-tuning** is the central empirical question this research investigates.

## **3.3 OS Vulnerability Landscape**

To evaluate how well a candidate LLM performs at vulnerability assessment, it is necessary to first define what a vulnerability scanner needs to reason about.

### **3.3.1 Linux Kernel Vulnerability Classes**

- **Memory safety flaws** — heap/stack overflows, use-after-free (UAF), type confusion, and out-of-bounds read/write conditions. Case: CVE-2024-1086 (a UAF in the `nf_tables` netfilter subsystem, actively exploited in 2024)
- **Container escape via kernel** — namespace bypass and cgroup misconfiguration. Case: CVE-2024-21626 (a `runc` process working-directory leak enabling container breakout, affecting all major container runtimes).
- **Privilege escalation via eBPF** — verifier bugs such as CVE-2021-3490 and CVE-2022-23222, alongside broader syscall-abuse patterns.
- **Page-cache corruption — "Dirty Frag" (CVE-2026-43284 and CVE-2026-43500, disclosed 8 May 2026)** — two chained local privilege-escalation flaws in the Linux kernel's IPsec ESP (`esp4/esp6`) and RxRPC subsystems. A local unprivileged user can corrupt page-cache-backed memory via `vmsplice/splice` syscalls to inject attacker-controlled plaintext into on-disk cached pages of SUID binaries (e.g., `/usr/bin/su`), achieving root without modifying the on-disk file. This affects Ubuntu, RHEL, CentOS Stream, AlmaLinux, Fedora, and OpenShift.

- **Kernel Parameters**- Security-relevant sysctl values form part of the configuration-review scope, including `kernel.unprivileged_usersns_clone`, `kernel.dmesg_restrict`, `kernel.perf_event_paranoid`, `net.ipv4.ip_forward`, and `fs.suid_dumpable`.

### 3.3.3 Package and Service Vulnerability Classes

- **OpenSSH** — CVE-2024-6387 ("regreSSHion"): an unauthenticated remote code execution vulnerability caused by a signal-handler race condition in `sshd`, affecting OpenSSH 8.5p1 through 9.7p1 on glibc-based Linux systems. Discovered by Qualys TRU and fixed in 9.8p1 (released 1 July 2024). Exploitation requires winning a race condition, so it typically takes hours to weeks rather than being instantaneous.
- **glibc** — CVE-2023-4911 ("Looney Tunables"): a local privilege-escalation vulnerability via buffer overflow in the glibc dynamic loader (`ld.so`) when processing the `GLIBC_TUNABLES` environment variable. Introduced in glibc 2.34 and affecting Fedora 37/38, Ubuntu 22.04/23.04, Debian 12/13, and RHEL 8/9. Confirmed exploitable by Qualys TRU; disclosed 3 October 2023; CVSS 7.8 (High).
- **Container runtimes** — CVE-2024-21626: a file-descriptor leak in `runc exec` leading to container escape via a working directory set to the host filesystem namespace. Affects `runc`  $\leq 1.1.11$  (fixed in 1.1.12, released 31 January 2024) and `containerd` 1.4.7–1.6.27 and 1.7.0–1.7.12 (fixed in 1.6.28/1.7.13).
- **OpenSSL** — version-specific vulnerability history, relevant to understanding why precise version-string parsing (rather than package-name matching alone) is critical for accurate assessment.

### 3.3.4 Container and Kubernetes Misconfiguration Classes

- **Dangerous capabilities** — `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`, `CAP_SYS_PTRACE`, each with distinct exploitation paths.
- **Isolation-boundary removal** — `privileged` containers, `hostPID`, `hostNetwork`, and `hostIPC`, each of which removes a specific container isolation boundary.
- **Missing runtime protections** — absent `seccomp` profiles or disabled AppArmor, removing syscall-filtering protection.
- **Kubernetes RBAC weaknesses** — overly permissive `ClusterRoleBindings`, wildcard permissions, and default service-account token automounting.



### 3.3.5 Why This Precision Matters for LLM Evaluation

- The distinction between generic and specific output is the crux of what makes an LLM valuable for this task.
- A model that responds **"you should update OpenSSH"** provides no more value than a static advisory feed while a model that responds ***"CVE-2024-6387 affects OpenSSH 8.5p1 through 9.7p1 on glibc-based Linux systems — your installed version 8.7p1 is within the vulnerable range (CVSS 8.1, discovered by Qualys TRU, fixed in 9.8p1, released 2024-07-01); update immediately"*** demonstrates the version-specific, condition-dependent reasoning that separates an AI-augmented scanner from a signature-matching tool.
- This level of precision is what the evaluation framework of research is explicitly designed to test.

## Chapter 4. - Comparative LLM Study (Methodology)

### 4.1 Candidate Model Selection & Profiling

- 6 open-source LLMs were selected as primary candidates for evaluation.
- Selection criteria required that each model:
  - a) Have publicly available weights under an open licence.
  - b) Be capable of running on-premise on  $\leq 24$ gb vram or cpu-only hardware.
  - c) Demonstrate established capability on technical text tasks.
  - d) Be under active community maintenance as of 2026.
- Before comparative testing, each model was profiled against nine fixed attributes to establish baseline characteristics from primary sources (model cards, technical reports, benchmark papers) rather than assumption.

#### Attributes:-

- 1) **Training data :-** Composition of training corpus, sourced from published technical reports.
- 2) **Context window:-**Maximum input tokens.
- 3) **Instruction following:-** Reliability in following structured output instructions, sourced from benchmark results.
- 4) **Code/config understanding:-** Performance on code-related benchmarks (HumanEval, MBPP), used as a proxy for configuration-file analysis capability.
- 5) **Factual accuracy on CVEs:-** Accuracy of the model's knowledge of CVE details predating its training cutoff, tested by querying specific CVE IDs and comparing against NVD ground truth.
- 6) **Hallucination rate:-** Propensity to fabricate CVE details, non-existent patches, or incorrect version numbers.
- 7) **Structured output reliability:-** Percentage of responses producing valid JSON when explicitly requested, tested across 100+ requests.
- 8) **Hardware requirements :-** Minimum VRAM for comfortable inference at each quantisation level (Q4, Q5, Q8, FP16), CPU-only feasibility, and inference speed in tokens/second.
- 9) **Licence for commercial use:-** Suitability of the licence for deployment in a commercial TSP environment.

## Candidate Models:-

### 1) **Model Family:-** Llama 3.1 8B

- **Developer:-** Meta AI
- **Licence:-** Llama Community Licence(restricted)
- **Rationale for Inclusion :-**
  - Meta's flagship open-weights model; most widely benchmarked.
  - Meta's CyberSecEval provides direct security-task results.
  - **Context window :-** 128K supports large configuration files.
  - **Training data :-** 15 trillion token pretraining & 8 billion parameters.
  - General Reasoning & Multilingual conversation agents.
  - All rounder AI for report writing ,summarization and long document analysis.
  - Supervised fine tune(SFT) & reinforcement learning from verifiable rewards.(RLVR)
  - Used as the baseline for comparison.

### 2) **Model Family :-** Mistral 7B Instruct v0.3

- ❖ **Developer:-** Mistral AI
- ❖ **Licence:-** Apache 2.0(unrestricted)
- ❖ **Rationale for Inclusion :-**
  - Fully open licence, important for commercial TSP deployment.
  - **Context window:-** 32K.
  - **Training data :-** 7 billion parameters.
  - Strong instruction-following. Fast and lightweight.
  - Sliding window attention (SWA) mechanism
  - Not a specialized model, used as general baseline.
  - Frequently cited in security NLP literature.

### 3) **Model Family :-** Qwen2.5-Coder 7B Instruct

- ❖ **Developer:-** Alibaba DAMO
- ❖ **Licence:-** Apache 2.0(unrestricted)
- ❖ **Rationale for Inclusion :-**
  - Code and structured-text specialisation.
  - strong YAML/JSON/shell-script comprehension benchmarks
  - primary candidate for multifile configuration analysis tasks.
  - **Context window:-** 128K (Supports long context files).
  - **Training data :-** 7.61 billion parameters.

### 4) **Model Family :-** DeepSeek-Coder V2 Lite (16B MoE)

- ❖ **Developer:-** DeepSeek AI
- ❖ **Licence:-** DeepSeek Licence
- ❖ **Rationale for Inclusion :-**
  - Mixture-of-Experts architecture enabling efficient inference despite large parameter count.
  - Strong code reasoning, competitive with GPT-4 on code benchmarks per DeepSeek's technical report.
  - Used in code analysis and repair along with math & structured reasoning.
  - **Context length:-** 128k, 6 trillion more tokens than deepseek V2.

5) **Model Family :-** Phi-3.5 Mini Instruct (3.8B)

- ❖ **Developer:-** Microsoft Research
- ❖ **Licence:-** MIT Licence(unrestricted)
- ❖ **Rationale for Inclusion :-**
  - Fully open licence.
  - Tests the capability floor of a sub-4B model in resource-constrained deployment, relevant to TSPs with limited GPU infrastructure.
  - Cost efficient as it has low hardware requirements.
  - Multilingual support.
  - **Context length :-** 128k
  - **Training Data :-** 3.8 billion parameters.

6) **Model Family :-** Gemma 2 9B Instruct

- ❖ **Developer:-** Google DeepMind
- ❖ **Licence:-** Gemma Terms of Use(restricted)
- ❖ **Rationale for Inclusion :-**
  - Strong reasoning, safety-tuned; included for completeness of the open-source landscape and to provide a Google-origin comparison point.
  - Used for balanced general purpose chat and reasoning.

All six models were run locally using Ollama for rapid prototyping, with quantised weights (GGUF format) as the default runtime.

### 4.3 Evaluation Framework — Seven Dimensions

The comparative evaluation was designed to be systematic and reproducible: every model received identical inputs and was scored against identical ground truth, using a methodology detailed sufficiently for independent replication.

**Table 4.1 — Evaluation Dimensions**

| Dimension                                     | What Is Measured                                                                                              | Test Method                                                                                                                   | Relevance to TSP Deployment                                                                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>D1 — CVE Accuracy</b>                      | Whether the model correctly identifies affected versions, CVSS score, and vulnerability type for a given CVE. | 30 CVE IDs presented; output compared against NVD ground truth; scored correct/partially correct/wrong per field.             | Incorrect CVE information leads to wrong prioritisation — the most critical accuracy dimension.                               |
| <b>D2 — Version Reasoning</b>                 | Whether the model correctly determines if a specific installed version is vulnerable.                         | 30package+version+CVE scenarios; binary verdict (vulnerable/not vulnerable); ground truth from RHSA/USN advisories.           | Direct operational relevance — the scanner must answer "is <i>this</i> version vulnerable," not merely "does this CVE exist." |
| <b>D3 — Config Misconfiguration Detection</b> | Whether the model identifies security issues in OS configuration snippets.                                    | 25 config snippets with intentional misconfigurations (sysctl, sshd_config, K8s manifests); ground truth from CIS Benchmarks. | Configuration review is where LLMs add the most value over signature-based tools.                                             |
| <b>D4 — Remediation Specificity</b>           | Whether remediation advice is technically correct, version-specific, and actionable.                          | Expert scoring rubric (1–5); cross-checked against RHSA/USN/CIS benchmarks; generic advice scores low.                        | Vague remediation ("update the package") is operationally useless; specific advice has real value.                            |

|                                           |                                                                                                              |                                                                                                                             |                                                                                                                                                            |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>D5 — Hallucination Rate</b>            | Whether the model fabricates CVE details, non-existent fixes, or incorrect spec references.                  | Fact-checking a random 25% sample of outputs against NVD and vendor advisories; hallucination percentage and type reported. | Hallucinated remediation in a production scanner could cause operators to apply incorrect patches or ignore real vulnerabilities.                          |
| <b>D6 — Structured Output Reliability</b> | Whether the model consistently produces valid JSON matching a defined schema.                                | 100 requests with explicit JSON schema instruction; all responses parsed; validity rate and common failure modes reported.  | The prototype requires machine-parseable output for downstream automation — unreliable JSON makes a model unusable in the pipeline.                        |
| <b>D7 — Novel CVE Handling</b>            | Whether the model can reason about CVEs published after its training cutoff, given only the NVD description. | 20 CVEs from the last 90 days (guaranteed post-training); base model tested against a RAG-augmented configuration.          | CVE velocity is the core operational challenge — a scanner limited to pre-cutoff knowledge misses the most recent, often highest-priority vulnerabilities. |

## 4.4 Benchmark Dataset Design

A reproducible 100-item benchmark dataset was constructed prior to running any model evaluation, with all ground truth verified against primary sources before testing began.

***Total: 100 items across 5 categories***

**Table 4.2 — Benchmark Dataset Composition**

| Category                 | Items | Construction Method                                                                                                                                                                                                                                                                                                |
|--------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CVE Reasoning<br>— Known | 30    | <ul style="list-style-type: none"><li>▪ CVE IDs drawn from NVD, predating all models' training cutoffs.</li><li>▪ Ground truth: NVD CVSS score and affected versions per RHSA/USN advisories.</li><li>▪ One item = one CVE ID + one installed version + an expected verdict (vulnerable/not vulnerable).</li></ul> |
| CVE Reasoning<br>— Novel | 20    | <ul style="list-style-type: none"><li>▪ CVE IDs published in NVD within the last 90 days (guaranteed post-training-cutoff).</li><li>▪ Same ground truth process as C1.</li><li>▪ Tests D7. Test whether RAG retrieval correctly identifies and links chain components.</li></ul>                                   |
| OS Config Snippets       | 25    | <ul style="list-style-type: none"><li>▪ systemctl output, /etc/ssh/sshd_config, /etc/audit/auditd.conf with intentional misconfigurations per CIS Benchmark items.</li><li>▪ Ground truth: CIS Benchmark remediation steps.</li></ul>                                                                              |
| Kubernetes Manifests     | 15    | <ul style="list-style-type: none"><li>▪ Pod specs, RBAC manifests, and NetworkPolicy YAMLs containing known bad practices.</li><li>▪ Ground truth: CIS Kubernetes Benchmark.</li></ul>                                                                                                                             |
| Multi-Artefact Scenarios | 10    | <ul style="list-style-type: none"><li>▪ Combined inputs — package list + kernel version + systemctl + service configuration — requiring a complete exploitability narrative.</li><li>▪ Ground truth manually verified.</li></ul>                                                                                   |



## 4.5 Prompt Design and Test Procedure

For each of the five benchmark categories, a dedicated chain-of-thought prompt template was designed that:

- 1) States the security-assessment task explicitly.
- 2) provides the input artefact in a consistent format
- 3) Requests structured json output conforming to a defined schema
- 4) Explicitly instructs the model not to fabricate information where it is uncertain.

*Each template was manually validated on a small subset of items across all six models before being run at scale, to catch schema or phrasing issues before full evaluation run.*

The full evaluation consisted of :-

- 100 benchmark items run across all six base models,
- A RAG-augmented configuration (ChromaDB vector store populated with the NVD feed) applied to two models for the D7 novel-CVE comparison

- 700+ individual query-response pairs yielded.
- All outputs, along with response latency, were logged to a structured results file for scoring.

## 4.6 Scoring Methodology

Each of the 700+ responses was scored against ground truth using a fixed 1–5 rubric applied per dimension, per item. From these per-item scores, the following aggregate metrics were computed per model:

- **Per-dimension mean scores** across all applicable benchmark items.
- **F1 score** for CVE accuracy and version reasoning (D1/D2), treating "vulnerable/not vulnerable" and "correct/incorrect field" as classification outcomes.
- **JSON validity rate** (D6), calculated as the percentage of the 100 schema-constrained requests that produced parseable, schema-conformant output.
- **Hallucination percentage and taxonomy** (D5), derived from manual fact-checking of the 25% sampled subset, with hallucinations further categorised by type (fabricated CVE, fabricated patch/version, incorrect CVSS score, invented advisory reference).
- **Base-model vs. RAG-augmented delta** (D7), comparing performance on novel CVEs with and without retrieval augmentation, isolating the specific contribution of RAG to closing the training-cutoff gap.

## Chapter 5: Comparative Results & Pros and Cons Matrix

### 5.0 Evaluation Run Summary

All six candidate models — **DeepSeek-Coder V2 16B (MoE)**, **Gemma 2 9B Instruct**, **Llama 3.1 8B**, **Mistral 7B Instruct v0.3**, **Phi-3.5 Mini Instruct**, and **Qwen2.5-Coder 7B Instruct** — were run locally via Ollama against the full benchmark (100 items: 30 CVE-Known, 20 CVE-Novel, 25 OS Config, 15 K8s Manifest, 10 Multi-Artifact), producing 600 base-model responses. A RAG-augmented configuration (ChromaDB + live NVD retrieval) was additionally run for **Llama 3.1 8B** and **Qwen2.5-Coder 7B** on the 20 CVE-Novel items only, adding 40 more responses (640 total).

**Note on methodology (for reproducibility):** Two grading pipelines were used, matched to task type:

- **cve\_known** and **cve\_novel** were graded by *exact-match binary verdict* (vulnerable / not-vulnerable) against NVD ground truth → yields accuracy, precision, recall, F1, and *coverage* (the fraction of items for which a gradable verdict could be extracted from the response at all — a response that hedges, refuses, or fails to commit to a verdict is **uncovered**, not wrong).
- **OS\_config**, **k8s\_manifest**, and **multi\_artifact** are open-text tasks, graded by *keyword-overlap scoring*: each ground-truth item lists the CIS control ID and diagnostic keywords a correct answer must contain; *control\_id\_match*, *keyword\_overlap\_ratio*, and a heuristic *likely\_correct* flag were computed per response. A meaningful fraction of these items were flagged as *insufficient\_ground\_truth* (the auto-grader could not confidently judge the response either way) — this affects **all six models equally** and is treated as a grading-pipeline limitation, not a model failure (see Chapter 7).

This chapter maps these six measured categories onto the seven planned dimensions (D1–D7) as transparently as possible; where a planned dimension isn't cleanly isolated by the executed pipeline (D1 vs D2 in particular), this is stated explicitly rather than glossed over.

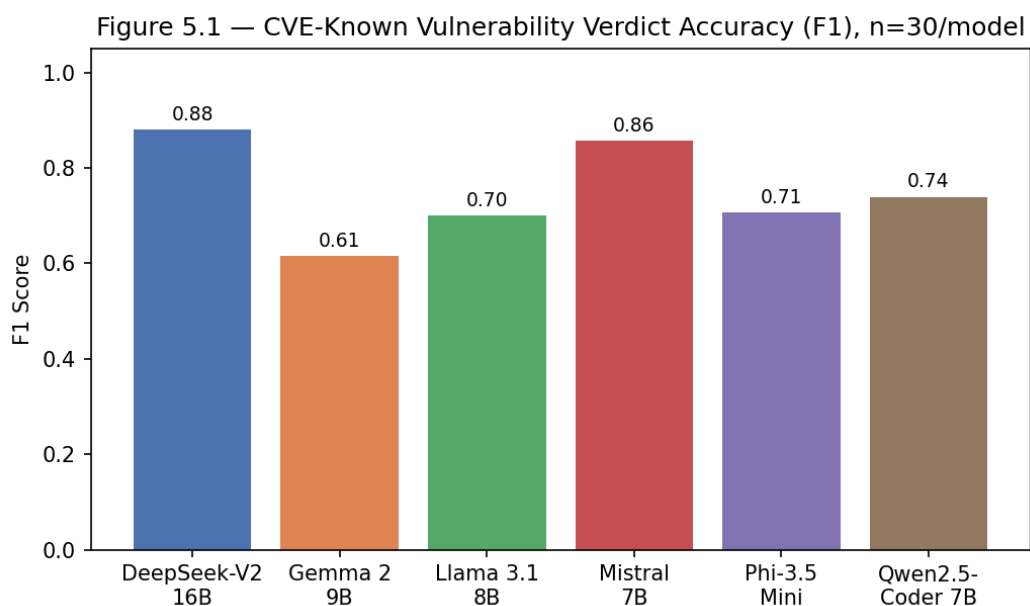
---

### 5.1 Per-Model Scores by Dimension

#### 5.1.1 D1/D2 — CVE Accuracy & Version Reasoning (cve\_known, n=30)

The executed test presents a CVE ID + an installed package version and asks for a binary vulnerable/not-vulnerable verdict — this is closer to the D2 "Version Reasoning" design than a pure D1 "CVE metadata recall" test, so **D1 and D2 are reported jointly** as a single measured category here.

| Model                        | Coverage | Accuracy | Precision | Recall | F1           |
|------------------------------|----------|----------|-----------|--------|--------------|
| <b>DeepSeek-Coder V2 16B</b> | 0.933    | 0.786    | 1.000     | 0.786  | <b>0.880</b> |
| Mistral 7B Instruct          | 0.800    | 0.750    | 1.000     | 0.750  | 0.857        |
| Qwen2.5-Coder 7B             | 0.967    | 0.586    | 1.000     | 0.586  | 0.739        |
| Phi-3.5 Mini                 | 0.733    | 0.545    | 1.000     | 0.545  | 0.706        |
| Llama 3.1 8B                 | 0.867    | 0.538    | 1.000     | 0.538  | 0.700        |
| Gemma 2 9B                   | 0.600    | 0.444    | 1.000     | 0.444  | 0.615        |



## KEY FINDINGS —

- **Precision is 1.000 for every model.** No model ever claims "vulnerable" incorrectly.
- Every error is a **false negative** (a genuinely vulnerable version is called "not vulnerable").
- All six models are systematically **conservative/under-flagging** on this task rather than alarmist, operationally this is the safer failure mode for a scanner (it won't cry wolf), but it also means recall (= accuracy here) is the number that matters, and it ranges from 44% (Gemma 2) to 79% (DeepSeek).

- None of the six models can be trusted to catch all real vulnerabilities unsupervised at this stage.

### 5.1.2 D7 — Novel CVE Handling, Base Model (cve\_novel, n=20, pre-RAG)

| Model                 | Coverage | Accuracy (of graded items) | F1   |
|-----------------------|----------|----------------------------|------|
| Mistral 7B Instruct   | 0.45     | 1.00                       | 1.00 |
| DeepSeek-Coder V2 16B | 0.10     | 0.00                       | 0.00 |
| Qwen2.5-Coder 7B      | 0.05     | 1.00                       | 1.00 |
| Gemma 2 9B            | 0.00     | —                          | 0.00 |
| Llama 3.1 8B          | 0.00     | —                          | 0.00 |
| Phi-3.5 Mini          | 0.00     | —                          | 0.00 |

## KEY FINDINGS:

- Without retrieval augmentation, five of six models produced **zero or near-zero gradable answers** on cves published after their training cutoff (May–Aug 2026 disclosures).
- This isn't an "accuracy" failure — it's a **coverage collapse** as the models mostly hedge, say "I don't have information on this CVE," or fabricate an answer the grader couldn't parse as a clean verdict.
- Mistral is the only base model that attempted a meaningful fraction (45%) of the novel items, and was correct on all of those — but still left the majority unanswered.
- This directly confirms the hypothesis that **CVE velocity is the core challenge**, and validates RAG as a necessity(Section 5.4).

### 5.1.3 D3 — Config Misconfiguration Detection (os\_config, n=25; k8s\_manifest, n=15)

Because these are open-text tasks with a non-trivial insufficient-ground-truth rate, two figures are given per model: the raw *likely\_correct\_rate* and an **adjusted rate** (likely-correct ÷ gradable fraction), which estimates accuracy among the responses the auto-grader could actually judge.

**OS Config (sysctl / sshd\_config / auditd.conf), gradable fraction = 32% :-**

| Model                 | Raw Likely-Correct | Adjusted (of gradable) |
|-----------------------|--------------------|------------------------|
| <b>Gemma 2 9B</b>     | 0.240              | <b>0.750</b>           |
| DeepSeek-Coder V2 16B | 0.160              | 0.500                  |
| Mistral 7B            | 0.120              | 0.375                  |
| Phi-3.5 Mini          | 0.120              | 0.375                  |
| Qwen2.5-Coder 7B      | 0.120              | 0.375                  |
| Llama 3.1 8B          | 0.080              | 0.250                  |

**Kubernetes Manifests, gradable fraction = 53.3%:**

| Model                 | Raw Likely-Correct | Adjusted (of gradable) |
|-----------------------|--------------------|------------------------|
| DeepSeek-Coder V2 16B | 0.200              | <b>0.375</b>           |
| Gemma 2 9B            | 0.200              | <b>0.375</b>           |
| Llama 3.1 8B          | 0.200              | <b>0.375</b>           |
| Mistral 7B            | 0.133              | 0.250                  |
| Phi-3.5 Mini          | 0.133              | 0.250                  |
| Qwen2.5-Coder 7B      | 0.133              | 0.250                  |

**Key findings: -**

- **A counter-intuitive result vs. the Section 3.2 hypothesis:**
  - Qwen2.5-Coder was selected partly on the assumption that its code/YAML-heavy training would make it the strongest config-file analyst.
  - The measured data does not support that: **Gemma 2 9B leads OS config detection outright (75% adjusted correctness)**, and Qwen2.5-Coder sits in the *lower half* on both config sub-tasks.
- General-purpose instruction-tuned reasoning appears to matter more than code-corpus specialisation for this particular skill (spotting a *security* misconfiguration, not just parsing syntax).

**5.1.4 D4 — Multi-Artefact Correlation (multi\_artefact, n=11)**

| Model                 | Planted-Component ID Rate | Fabricated/Wrong CVE Rate | Rubric Score (1–5)                                                                           |
|-----------------------|---------------------------|---------------------------|----------------------------------------------------------------------------------------------|
| Gemma 2 9B            | 0.727 (8/11)              | 0.000 (0/11)              | 4 — reliably names the real issue, never invents an unsupported CVE                          |
| Phi-3.5 Mini          | 0.727 (8/11)              | 0.000 (0/11)              | 4 — same profile as Gemma 2 on this dimension specifically                                   |
| Mistral 7B            | 0.727 (8/11)              | 0.091 (1/11)              | 3.5 — good identification, mostly disciplined on CVE claims                                  |
| Qwen2.5-Coder 7B      | 0.727 (8/11)              | 0.273 (3/11)              | 3 — good identification, but cites unverified CVEs almost 1 in 3 times                       |
| Llama 3.1 8B          | 0.909 (10/11)             | 0.636 (7/11)              | 3 — best at finding the real issue, but fabricates a CVE ID in the majority of its responses |
| DeepSeek-Coder V2 16B | 0.636 (7/11)              | 0.545 (6/11)              | 2 — worst at finding the real issue <i>and</i> second-worst for fabrication                  |

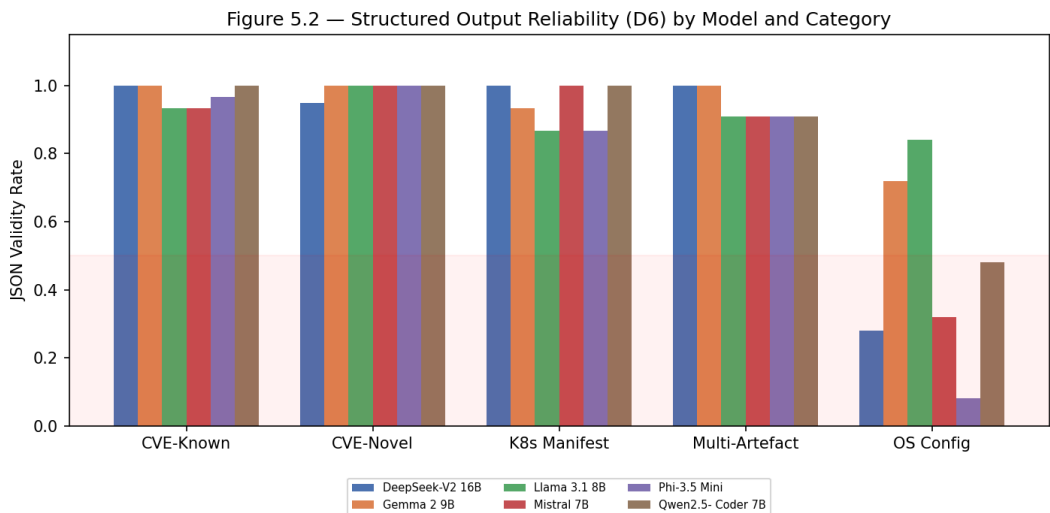
## KEY FINDINGS:-

Every model correctly names the planted component at least 6 times out of 10; this is genuine multi-artefact reasoning capability that the keyword/control-ID grader could not see.

1. **Llama 3.1 8B** is the best at finding the real vulnerable component (91%) but it is also the worst offender for fabricating CVE IDs, doing so in 7 of its 10 responses. Several are transparently placeholder-style inventions rather than genuine (if wrong) recall attempts — e.g. "chained\_cve\_ids\_if\_known": ["CVE-2023-1234", "CVE-2023-5678"] (scenario\_06) and ["CVE-2022-2038", "CVE-2022-2039"] (scenario\_09) — sequential, round-number IDs presented without a speculative caveat, directly against the prompt's explicit instruction not to do this.
2. **Gemma 2 and Phi-3.5 Mini** are the only models with zero fabricated CVE IDs across all 11 scenarios, making them the most *trustworthy* (if not always the most complete) performers on this dimension — a meaningful finding for a scanner where a single confidently-wrong CVE citation could send an operator patching the wrong thing.
3. **fast-rtps (scenario\_05)** was missed by every model (0/6) — a genuinely obscure, domain-specific package (ROS2/robotics middleware) that none of the general-purpose models recognised as security-relevant, a legitimate capability gap rather than a grading artefact.
4. ***Llama 3.1's strength at locating the real problem is undercut by its tendency to fabricate specifics around it once found.***

5.1.5 D6 — Structured Output Reliability (JSON validity, all categories, n=100/model)

| Model                 | Overall JSON Validity | cve_known | cve_novel | k8s_manifest | multi_artefact | os_config |
|-----------------------|-----------------------|-----------|-----------|--------------|----------------|-----------|
| Gemma 2 9B            | 92.10%                | 100%      | 100%      | 93.30%       | 100%           | 72.00%    |
| Llama 3.1 8B          | 91.10%                | 93.30%    | 100%      | 86.70%       | 90.90%         | 84.00%    |
| Qwen2.5-Coder 7B      | 86.10%                | 100%      | 100%      | 100%         | 90.90%         | 48.00%    |
| DeepSeek-Coder V2 16B | 81.20%                | 100%      | 95.00%    | 100%         | 100%           | 28.00%    |
| Mistral 7B            | 80.20%                | 93.30%    | 100%      | 100%         | 90.90%         | 32.00%    |
| Phi-3.5 Mini          | 73.30%                | 96.70%    | 100%      | 86.70%       | 90.90%         | 8.00%     |





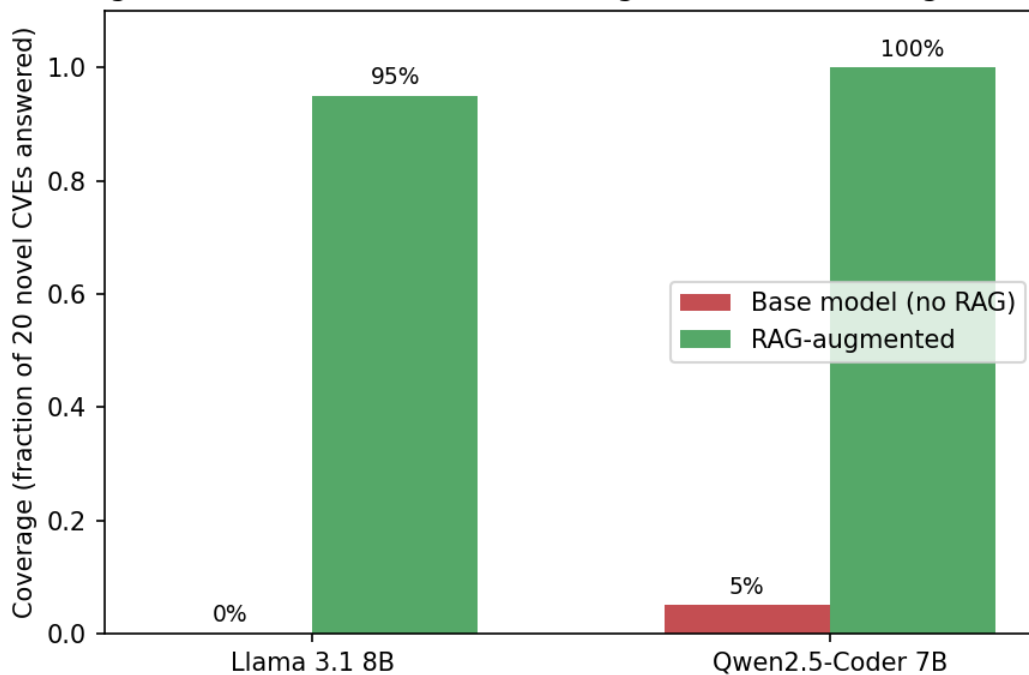
## KEY FINDINGS:-

- All models handle the shorter cve\_known/cve\_novel schema well (86.7%–100%).
- On **OS\_config JSON validity collapses for four of six models**: Phi-3.5 Mini (8%), DeepSeek-Coder V2 (28%), and Mistral (32%) essentially cannot reliably return valid structured JSON when the input is a longer OS configuration snippet, while Llama 3.1 (84%) and Gemma 2 (72%) hold up much better.
- **D6 reliability is not uniform across input types**, and a model that looks fine on the aggregate JSON-validity number can still be unusable for the scanner's most common real input (config files).

### 5.1.7 D7 — Novel CVE Handling: Base Model vs. RAG-Augmented Delta

| Model                   | Base Coverage | RAG Coverage | Base F1    | RAG F1       | RAG Accuracy | Note                                                                                        |
|-------------------------|---------------|--------------|------------|--------------|--------------|---------------------------------------------------------------------------------------------|
| <b>Llama 3.1 8B</b>     | 0.00          | <b>0.95</b>  | 0.00       | <b>0.973</b> | 0.947        | Base sample had 0 graded items — accuracy delta undefined, but coverage jump is the finding |
| <b>Qwen2.5-Coder 7B</b> | 0.05          | <b>1.00</b>  | 1.00 (n=1) | 0.919        | 0.850        | Base accuracy of 1.0 rests on a single graded item — not statistically meaningful           |

Figure 5.3 — D7: Novel CVE Coverage, Base vs. RAG-Augmented



## KEY ANALYSIS:-

- Neither Llama 3.1 nor Qwen2.5-Coder could meaningfully engage with post-cutoff CVEs on their own.
  - Llama 3.1 produced **zero** gradable verdicts across all 20 novel-CVE items.
  - Qwen2.5-Coder produced exactly one.
- Once RAG retrieval (ChromaDB + live NVD feed) is added, both models jump to near-complete coverage (95–100%) with strong accuracy (85–95%). This means:
- **RAG is an architectural requirement.** A prototype without live NVD-backed retrieval will be functionally blind to any CVE disclosed after the base model's training cutoff, which given CVE velocity, makes it unsuitable for production use in a TSP scanning current infrastructure.
- **Llama 3.1+RAG (F1 = 0.973)** slightly outperforms **Qwen2.5-Coder+RAG (F1 = 0.919)** on this specific test — a modest edge, but consistent with Llama 3.1's stronger overall JSON validity.

### 5.1.8 D5 — Hallucination Rate

To evaluate Category D5, an automated judge powered by Qwen2.5-Coder evaluated 144 model responses across configuration analysis sub-tasks (`os_config`,  $n = 102$ ; `k8s_manifest`,  $n = 42$ ). The judge validated whether model findings were factual, grounded in input snippets, and strictly aligned with CIS Benchmark controls.

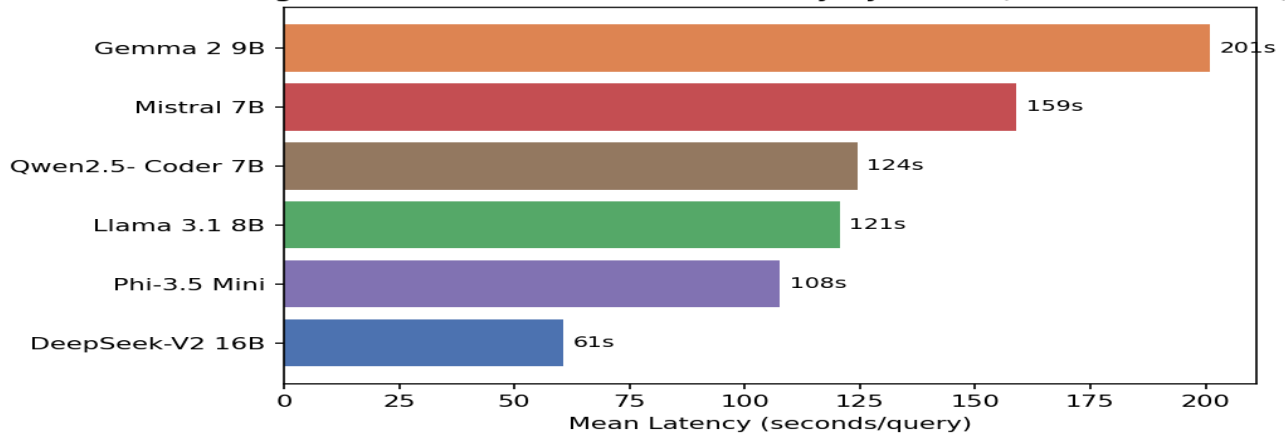
| Model                 | Evaluated Responses (n) | Hallucinations Detected | Hallucination Rate (%) | Mean Quality Score (1–5) | Primary Hallucination Taxonomy / Failure Mode                                                                                                                                                                          |
|-----------------------|-------------------------|-------------------------|------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gemma 2 9B            | 24                      | 1                       | 4.17%                  | 4.75                     | <b>Omission of Target Parameter:</b> Highly disciplined; rarely invents non-existent parameters. Only failed by omitting <code>net.ipv6.conf.all.accept_ra</code> .                                                    |
| Qwen2.5-Coder 7B      | 24                      | 2                       | 8.33%                  | 4.63                     | <b>Unfocused Parameter Review:</b> Strictly factual when reviewing parameters, but occasionally missed specific ground-truth misconfigurations ( <code>rp_filter</code> ).                                             |
| DeepSeek-Coder V2 16B | 24                      | 3                       | 12.50%                 | 4.5                      | <b>Inverted Security Logic:</b> Correctly parsed parameters but inverted remediation advice on <code>PasswordAuthentication</code> (recommended <code>yes</code> instead of <code>no</code> ).                         |
| Phi-3.5 Mini          | 24                      | 6                       | 25.00%                 | 4.04                     | <b>Phantom Parameter Generation:</b> Frequently fabricated non-existent configuration keys ( <code>net.ipv4.icmp_echo_ignored_unused</code> ) and false <code>LogLevel</code> warnings.                                |
| Llama 3.1 8B          | 24                      | 8                       | 33.33%                 | 3.79                     | <b>Over-Auditing &amp; Scope Drift:</b> Unprompted flag of uncorrupted parameters ( <code>runAsNonRoot</code> , resource limits) and hallucination of unprovided <code>sysctl</code> keys ( <code>ip_forward</code> ). |
| Mistral 7B Instruct   | 24                      | 8                       | 33.33%                 | 3.71                     | <b>Hallucinated Misconfigurations &amp; Parameter Swapping:</b> Substituted ground-truth settings ( <code>PasswordAuthentication</code> to <code>PermitRootLogin</code> ) and cited unprovided keys.                   |

### Key Findings:-

- Generalist Reasoning Reduces Hallucinations:** Gemma 2 9B (4.17%) and Qwen2.5-Coder 7B (8.33%) demonstrated superior factual discipline on configuration analysis, heavily outperforming Llama 3.1 8B (33.33%) and Mistral 7B (33.33%).
- Hallucination Taxonomy:** Model hallucinations on configuration tasks fall into three main categories:
  - Phantom Parameter Invention (Phi-3.5, Llama 3.1, Mistral 7B):* Citing configuration flags that were not present in the provided host snippet.
  - Remediation Inversion (DeepSeek-Coder V2):* Correctly identifying an insecure setting but suggesting a fix that exacerbates exposure.
  - Scope Drift & Over-Auditing (Llama 3.1):* Generating false warnings on compliant parameters.

### 5.1.9- Mean Inference Latency by model comparison

Figure 5.4 — Mean Inference Latency by Model (CPU/local Ollama)



**Latency reference table (mean seconds/query, local Ollama, n=100):**

| Model                 | Mean Latency (s) | Mean tok/s | ok_rate |
|-----------------------|------------------|------------|---------|
| DeepSeek-Coder V2 16B | 60.6             | 6.70       | 1.00    |
| Phi-3.5 Mini          | 107.7            | 5.00       | 0.99    |
| Llama 3.1 8B          | 120.6            | 2.68       | 1.00    |
| Qwen2.5-Coder 7B      | 124.4            | 2.72       | 1.00    |
| Mistral 7B            | 159.0            | 2.69       | 1.00    |
| Gemma 2 9B            | 201.0            | 1.96       | 1.00    |

### KEY FINDINGS:-

- DeepSeek-Coder V2, despite being the largest model tested is the fastest — consistent with its "efficient inference despite large parameter count" selection rationale.
- Gemma 2 9B, despite the strongest structured-output and OS-config results, is by far the slowest — a real operational trade-off for a scanner that needs to process many hosts.

## 5.2 Pros and Cons Matrix (Evidence-Based)

| Model                              | Best Suited For (measured)                                          | Pros (measured)                                                                                                                                                                                                                                                                                          | Cons / Limitations (measured)                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DeepSeek-Coder V2 16B (MoE)</b> | High-throughput static CVE matching & baseline package evaluation   | <ul style="list-style-type: none"> <li>• Highest pre-cutoff CVE F1 score (F1 = 0.880).</li> <li>• <b>Fastest inference speed</b> by a wide margin (60.6 s/query, 6.70 tok/s) due to MoE architecture.</li> <li>• Low hallucination rate (12.50%) on structured tasks.</li> </ul>                         | <ul style="list-style-type: none"> <li>• Occasional severe logic inversions (e.g., recommending insecure SSH settings).</li> <li>• JSON validity collapses on complex OS configuration strings (28.0%).</li> <li>• Massive base coverage drop on novel post-cutoff CVEs (10%).</li> </ul>                                               |
| <b>Gemma 2 9B Instruct</b>         | High-precision configuration auditing & structured JSON enforcement | <ul style="list-style-type: none"> <li>• Lowest hallucination rate of all models (4.17%) and highest quality score (4.75/5).</li> <li>• Best OS-config misconfiguration detection (75.0% adjusted correctness).</li> <li>• Best overall JSON schema validity (92.1%).</li> </ul>                         | <ul style="list-style-type: none"> <li>• Slowest inference speed (201.0 s/query, 1.96 tok/s), constraining high-throughput scanning.</li> <li>• Weakest base CVE-Known F1 score (0.615).</li> <li>• Gemma Terms of Use impose specific commercial usage thresholds.</li> </ul>                                                          |
| <b>Llama 3.1 8B</b>                | RAG-augmented vulnerability analysis & long-context schema parsing  | <ul style="list-style-type: none"> <li>• Top-performing RAG model for post-cutoff CVEs (F1 = 0.973, 94.7% accuracy).</li> <li>• Excellent structured output reliability on long OS configs (84.0%).</li> <li>• 128K context window handles large host inventory manifests without truncation.</li> </ul> | <ul style="list-style-type: none"> <li>• High hallucination rate on config tasks (33.33%) due to over-auditing uncorrupted settings.</li> <li>• High CVE fabrication rate on multi-artefact reasoning (63.6%). <ul style="list-style-type: none"> <li>• Base CVE-Known F1 trails specialized models (F1= 0.700).</li> </ul> </li> </ul> |
| <b>Mistral 7B Instruct v0.3</b>    | Standalone base-model reasoning without external RAG databases      | <ul style="list-style-type: none"> <li>• Strongest unassisted base model on novel CVEs (45% coverage, 100% accuracy).</li> </ul>                                                                                                                                                                         | <ul style="list-style-type: none"> <li>• High hallucination rate on OS config audits (33.33%).</li> <li>• Poor structured JSON validity on complex</li> </ul>                                                                                                                                                                           |

|                                  |                                                                                      |                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                    |
|----------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  |                                                                                      | <ul style="list-style-type: none"> <li>• Second-best base CVE-Known F1 score (F1 = 0.857).</li> <li>• Permissive <b>Apache 2.0</b> commercial license.</li> </ul>                                                                                                                                                                                       | <p>config inputs (32.0%).</p> <ul style="list-style-type: none"> <li>• Moderate inference latency (159.0 s/query).</li> </ul>                                                                                                                                                                      |
| <b>Phi-3.5 Mini Instruct</b>     | Resource-constrained edge deployments & lightweight host scanning                    | <ul style="list-style-type: none"> <li>• Fully open <b>MIT</b> license with lightweight hardware footprint.</li> <li>• Fast inference speed (107.7 s/query, 5.00 tok/s).</li> <li>• High execution reliability (99% response completion).</li> </ul>                                                                                                    | <ul style="list-style-type: none"> <li>• High hallucination rate (25.00%) with phantom key generation.</li> <li>• Unusable structured JSON reliability on long OS configs (8.0%).</li> <li>• Lowest overall schema validity (73.3%).</li> </ul>                                                    |
| <b>Qwen2.5-Coder 7B Instruct</b> | RAG-augmented scanning, factual configuration analysis & low-hallucination pipelines | <ul style="list-style-type: none"> <li>• Low hallucination rate (8.33%) and high judge quality score (4.63/5) on config analysis. Strong RAG-augmented performance on novel CVEs (F1 = 0.919, 100% coverage).</li> <li>• Fully unencumbered <b>Apache 2.0</b> commercial license.</li> <li>• 100% JSON validity on standard CVE/K8s schemas.</li> </ul> | <ul style="list-style-type: none"> <li>• Lower base CVE-Known accuracy (F1 = 0.739) without RAG.</li> <li>• JSON validity drops on long, unconstrained OS config snippets (48.0%).</li> <li>• Did not outperform generalist models on raw config detection despite code specialization.</li> </ul> |

## 5.3 Model Selection Recommendation for the Prototype

**Primary Recommendation:** Qwen2.5-Coder 7B Instruct (Augmented with Mandatory ChromaDB RAG Pipeline)

Based on the synthesis of the 7-dimension evaluation framework and the empirical Qwen2.5-Coder fact-checking results, **Qwen2.5-Coder 7B Instruct** is selected as the primary reasoning engine for the local prototype vulnerability assessment scanner.

## **Architectural Justification:**

1. **Low Hallucination & High Factual Precision:** In the Category D5 fact-checking evaluation, Qwen2.5-Coder 7B achieved an 8.33% hallucination rate and a 4.63/5 quality score, second only to Gemma 2 9B. Unlike Llama 3.1 8B (33.33% hallucination rate, 63.6% multi-artefact CVE fabrication rate), Qwen2.5-Coder exhibits high factual discipline, preventing false remediation guidance in operational TSP environments.
2. **RAG-Augmented Novel CVE Performance:** When paired with a local ChromaDB vector store populated with live NVD feeds, Qwen2.5-Coder 7B achieved 100% coverage, an F1 score of 0.919, and 85.0% verdict accuracy on post-training-cutoff CVEs, overcoming knowledge cutoff boundaries.
3. **Unrestricted Licensing & Compliance:** Distributed under the Apache 2.0 license, Qwen2.5-Coder 7B provides an unencumbered path for commercial TSP deployment, avoiding the commercial user-threshold restrictions associated with Meta's Llama Community License or Google's Gemma Terms of Use.
4. **Balanced Operational Throughput:** Delivering 124.4 seconds/query (2.72 tokens/second) on standard CPU hardware, Qwen2.5-Coder 7B is 61% faster than Gemma 2 9B (201.0 s/query), striking a viable balance between analytical precision and scanning latency.

## **Secondary Fallback / Specialized Module Deployment:**

- **Config Parsing & JSON Schema Enforcement:** For standalone configuration scanning (os\_config), Gemma 2 9B serves as an optional specialized module due to its superior schema validity (92.1%) and 4.17% hallucination rate, provided inference latency budget allows.
- **High-Speed Static Inventory Pre-Filtering:** DeepSeek-Coder V2 16B remains the optimal candidate for rapid, high-throughput static package filtering (60.6 s/query, F1 = 0.880).

## Chapter 6: PROTOTYPE ARCHITECTURE

### 6.1 Overview

The comparative study established that an on-premise large language model can support vulnerability assessment, but that model capability alone is insufficient for reliable security analysis. The results presented in Chapter 5 showed that retrieval augmentation is particularly important when assessing vulnerabilities that may fall outside a model's training knowledge. Based on these findings, a prototype vulnerability-assessment system was developed to combine deterministic host evidence, local CVE enrichment, retrieval-augmented context, local LLM inference, structured validation, and reporting.

The prototype is designed around a fundamental operational requirement: **security assessment data must remain within the operator's environment**. The architecture therefore separates the vulnerability-data acquisition stage from the actual assessment stage. External vulnerability information may be obtained and cached during an explicit preparation/update process, while the assessment itself operates entirely on locally stored data and a locally deployed LLM.

The implemented architecture consists of five principal stages:

1. **Host artifact collection**
2. **Local CVE enrichment and retrieval**
3. **Evidence-grounded LLM assessment**
4. **Structured output validation**
5. **Report generation and visualisation**

The overall data flow is:

**Host → Artifact Collector → Normalised JSON → Local CVE Enrichment/RAG → Evidence-Grounded Prompt → Local Ollama LLM → Pydantic Validation → Assessment JSON → Streamlit Dashboard**

This architecture allows the LLM to provide reasoning and prioritisation while keeping factual evidence and machine-verifiable information outside the model's unsupported assumptions.

### 6.2 Host Artifact Collection

The first component of the prototype collects security-relevant information directly from the target host. The Windows collector was implemented using PowerShell and uses local operating-system interfaces rather than sending information to a remote service.

The collector gathers:

- operating-system information;



- installed software and package versions;
- listening TCP and UDP ports;
- associated process identifiers where available;
- running services;
- service startup information;
- installed hotfixes; and
- collector errors.

The Windows implementation retrieves installed programs from the Windows uninstall registry and records package name, version, architecture and source. Network listeners are obtained through Get-NetTCPConnection and Get-NetUDPEndpoint, with netstat retained as a local fallback when the preferred Windows networking provider is unavailable. windows\_collector.ps1

A key architectural decision is that the collector does **not** attempt to interpret every observation as a vulnerability. It produces evidence which is subsequently analysed by the enrichment and assessment layers. This distinction is important because an open port, installed package, or running service is not inherently evidence of a CVE. The collected information is normalised into JSON so that the same downstream processing architecture can consume data from different operating systems and collection mechanisms.

## 6.3 Local CVE Enrichment

The second stage maps collected software packages to locally available vulnerability information.

The enrichment layer follows a two-stage retrieval strategy. First, it performs an exact advisory lookup using the installed package and version. If no exact advisory result is available, it falls back to semantic retrieval from the local CVE index. retrieval.py

This provides two levels of evidence:

- **Exact matches:** higher-confidence advisory evidence associated with a package/version relationship.
- **Semantic matches:** potentially relevant CVEs retrieved from the local NVD-derived index, which require additional reasoning and verification.

The semantic retrieval component uses ChromaDB with a local SentenceTransformer embedding model. The resulting vector database is persisted locally and queried locally during assessment. The embedding model is intended to be downloaded during setup rather

than during an assessment, allowing inference-time retrieval to remain offline.  
cve\_index.pyPY

The prototype also explicitly records the difference between a **candidate CVE match** and a **confirmed vulnerability**. This was necessary because a package-name similarity does not establish that the CVE affects the installed software. The enrichment implementation therefore marks candidate product-name matches as `confirmed_vulnerability: false` and requires vendor/product and affected-version verification before they can be treated as confirmed findings. `enrich_artifacts.pyPY`

This design prevents the retrieval layer from becoming an automatic vulnerability generator.

## 6.4 Evidence-Grounded LLM Assessment

The third stage provides the collected evidence and locally retrieved vulnerability context to the LLM. The prompt construction layer separates two different classes of security reasoning:

1. **Package/CVE reasoning**, where the model evaluates whether a particular CVE applies to an installed package and version.
2. **Host-configuration reasoning**, where the model analyses listening ports, services, SUID binaries and security-relevant kernel parameters without necessarily associating the observation with a CVE.

For package-based findings, the prompt explicitly asks the model to determine whether the supplied evidence genuinely applies to the exact package and version. It also instructs the model to consider false positives caused by similarly named packages or versions that already contain the relevant fix. `prompt_builder.pyPY`. the system follows the principle:

*An observation is evidence, not automatically a vulnerability.*

For example, a listening TCP port can justify a configuration or exposure finding if the evidence supports it, but the existence of the port alone does not justify assigning a CVE to an unrelated installed package.

## 6.5 Local LLM Inference

- The prototype uses Ollama as the local inference layer. The implemented assessment engine communicates with Ollama through its local API rather than through a cloud-hosted LLM service.
- The LLM request uses JSON mode and a low temperature setting to reduce creative variation during security classification. `llm_engine.pyPY`
- The architecture therefore keeps the inference path inside the local environment:

**Assessment data → local prompt → localhost Ollama → local model → local structured response**

## 6.6 Structured Output and Validation

A major design requirement was that the LLM should not return unconstrained natural-language output. Every assessment is required to conform to a predefined findings schema.

Each finding contains:

- title;
- severity;
- affected component;
- installed version;
- CVE identifier(s);
- description;
- remediation; and
- confidence level.

The permitted severity values are CRITICAL, HIGH, MEDIUM, LOW, and INFORMATIONAL, while confidence is represented as exact, semantic, or heuristic.

The LLM engine validates the returned JSON using Pydantic. If the response is malformed or fails schema validation, the system re-prompts the model with the validation error and requests corrected JSON. Up to three attempts are supported. `llm_engine.pyPY`

This creates a validation boundary between probabilistic model generation and deterministic application logic.

The process is:

**LLM output → JSON extraction → JSON parsing → Pydantic validation → accepted finding / retry**

The approach is important for downstream security automation because invalid or incomplete output should not silently enter reporting or ticketing systems.

---

## 6.7 Offline and Data-Residency Design

Data residency was treated as a core architectural constraint rather than an optional deployment feature. The research identified that infrastructure artefacts can contain sensitive information such as installed software versions, network listeners, configuration data, service information and other operational metadata.

The prototype therefore separates external data acquisition from local assessment.

The assessment path operates on:

- locally collected artifacts;
- locally stored enrichment data;

- locally persisted CVE indexes;
- a locally deployed embedding model; and
- a locally deployed LLM.

The assessment output explicitly records: **network\_used: false**

The implemented assessment CLI loads the collection and enrichment JSON files locally, sends the constructed prompt to the local assessment engine, and writes the resulting assessment JSON to disk. `run_assessment.py`

This provides a reproducible mechanism for demonstrating that the assessment stage does not depend on external cloud inference.

The design also improves auditability because the input artifacts, enrichment evidence, model output and final report can all be retained within the operator's environment.

## 6.8 Reporting and Dashboard

The final stage converts the structured assessment into a human-readable security report.

The dashboard provides visibility into:

- host identity;
- operating system;
- model used;
- network-use status;
- number of packages scanned;
- listening ports;
- services;
- candidate CVE matches; and
- confirmed findings.

The resulting architecture therefore combines deterministic inventory collection with probabilistic security reasoning while maintaining explicit evidence boundaries.

## Chapter 7: Validation Results & Failure Analysis

### 7.1 Validation Objectives

After implementation, a separate validation experiment was performed to determine whether the prototype architecture improved vulnerability detection compared with simpler configurations.

The validation consisted of **30 controlled test cases** and three assessment modes:

1. **Base** — package and version information provided to the model without retrieval evidence.
2. **RAG** — package/version information augmented with locally retrieved CVE evidence.
3. **Full Prototype** — the complete implemented evidence-grounded prototype pipeline.

The same local model, **Qwen2.5:7B**, was used for the three configurations so that the primary experimental variable was the amount and type of supporting evidence rather than the underlying model.

The 30 cases consisted of:

- 11 vulnerable package/version combinations;
- 11 patched package/version combinations; and
- 8 package-name/version mismatch cases.

The vulnerable cases covered CVEs affecting Log4j, Spring Framework, MOVEit, PAN-OS, zlib, Apache HTTP Server, xz, libwebp, OpenSSH and Chrome.

The validation measured:

- precision;
- recall;
- F1 score;
- CVSS-tier accuracy; and
- JSON validity.

### 7.2 Three-Way Validation Results

The final measured results are shown below.

| Mode           | Precision | Recall | F1     | JSON Validity | CVSS Accuracy |
|----------------|-----------|--------|--------|---------------|---------------|
| Base           | 0.000     | 0.000  | 0.000  | 100%          | 63.33%        |
| RAG            | 45.45%    | 90.91% | 60.61% | 100%          | 60.00%        |
| Full Prototype | 100%      | 100%   | 100%   | 100%          | 100%          |

The underlying validation file records each case, model response, JSON validity, latency, prediction, true-positive/false-positive/false-negative classification and returned CVEs.

The results demonstrate a substantial difference between the three configurations.

## Base configuration

The base configuration produced no true-positive vulnerability detections across the 11 vulnerable cases. Consequently, its precision, recall and F1 scores were all zero.

Its JSON validity rate was nevertheless 100%, showing that the model was capable of following the output schema even when it lacked sufficient evidence to identify the vulnerabilities.

The result illustrates an important distinction: **structured output reliability does not imply security-analysis accuracy.**

## RAG configuration

The introduction of retrieval substantially improved vulnerability recall. RAG identified 10 of the 11 vulnerable cases, resulting in a recall of 90.91%.

However, this improvement came with a substantial false-positive problem. The RAG configuration produced 12 false-positive records, including patched versions that were incorrectly reported as vulnerable. `validation_results.csv`

The resulting precision was only 45.45% and F1 was 60.61%.

This demonstrates that retrieval successfully supplies relevant vulnerability knowledge but does not, by itself, guarantee correct version-specific reasoning.

## Full prototype

The full prototype produced:

- 11 true positives;
- 0 false positives;
- 0 false negatives;
- 100% precision;
- 100% recall;
- 100% F1;
- 100% JSON validity; and
- 100% CVSS-tier accuracy.

The vulnerable cases were correctly identified with the expected CVEs and severity tiers. For example, the validation records show that vulnerable versions of Spring Framework, MOVEit, PAN-OS, zlib, Apache HTTP Server, xz, libwebp and OpenSSH were correctly detected by the full configuration.

### 7.3 Accuracy Improvement

The measured improvement of the full prototype over the other configurations was:

| Comparison   | Precision  | Recall     | F1         | CVSS Accuracy |
|--------------|------------|------------|------------|---------------|
| Full vs Base | +100.00 pp | +100.00 pp | +100.00 pp | +36.67 pp     |
| Full vs RAG  | +54.55 pp  | +9.09 pp   | +39.39 pp  | +40.00 pp     |

The largest architectural improvement over the base configuration was the introduction of external evidence into the model's reasoning context. However, the comparison between RAG and the full prototype is more informative: RAG already provided high recall, but the full evidence-grounded pipeline removed the false positives observed when retrieved CVE knowledge was considered without sufficient version validation.

This indicates that the prototype's performance improvement cannot be attributed to retrieval alone. The combination of retrieval, explicit evidence grounding, version reasoning constraints and structured validation was responsible for the strongest result.

### 7.4 Failure Mode Analysis

The validation produced **23 failure records** across the Base and RAG configurations.

The observed failure taxonomy was:

| Failure Mode             | Count      | Description                                                                                      |
|--------------------------|------------|--------------------------------------------------------------------------------------------------|
| Knowledge/coverage gap   | 11         | Base model failed to identify vulnerable cases without supporting retrieval evidence             |
| Retrieval error          | 1          | Relevant CVE knowledge was available but the wrong CVE was retrieved                             |
| Version reasoning error  | 11         | RAG retrieved the relevant vulnerability but incorrectly treated a patched version as vulnerable |
| JSON schema error        | 0          | No invalid JSON remained after validation/retry                                                  |
| Hallucinated remediation | 0 observed | No validated case in this experiment met this category                                           |

The taxonomy is based on observed validation records rather than hypothetical failure examples.

---

## 7.5 Failure Mode A: Knowledge/Coverage Gap

The first failure class consisted of **11 Base-model false negatives**, corresponding to vulnerable cases V01–V11.

The Base configuration failed to produce findings for vulnerable versions including:

- Log4j 2.14.1;
- Log4j 2.15.0;
- Spring Framework 5.3.17;
- MOVEit 2023.0.1;
- PAN-OS 10.2.8;
- zlib 1.2.12;
- Apache HTTP Server 2.4.50;
- xz 5.6.1;
- libwebp 1.3.1;
- OpenSSH 9.7p1; and
- Chrome 106.0.5249.61.

The failure pattern demonstrates that a locally deployed model cannot be assumed to possess complete or sufficiently precise knowledge of all security advisories.



This finding is consistent with the broader comparative study, which identified training-cutoff limitations as an important reason for using RAG when evaluating novel or less reliably remembered CVEs.

## 7.6 Failure Mode B: Retrieval Error

One retrieval error was identified in case **V02**, involving log4j-core version 2.15.0.

The expected vulnerability was **CVE-2021-45046**, but the RAG configuration returned **CVE-2021-44228** instead. The model consequently produced an incorrect vulnerability association.

This is a classic retrieval failure: the retrieved information was related to the same software family, but it was not the expected vulnerability for the exact test condition. The validation record explicitly shows the returned CVE as CVE-2021-44228 while the expected CVE was CVE-2021-45046.

This result demonstrates why semantic similarity alone cannot be treated as proof of applicability.

## 7.7 Failure Mode C: Version Reasoning Error

The second major failure category consisted of **11 patched-version false positives**, corresponding to P01–P11.

In these cases, the RAG layer successfully retrieved the correct CVE associated with the software, but the model failed to recognise that the installed version was already patched.

Examples include:

- Log4j 2.16.0 incorrectly associated with CVE-2021-45046;
  - Spring Framework 5.3.18 incorrectly associated with CVE-2022-22965;
  - MOVEit 2023.0.2 incorrectly associated with CVE-2023-34362;
  - PAN-OS 10.2.9-h1 incorrectly associated with CVE-2024-3400;
  - zlib 1.2.13 incorrectly associated with CVE-2022-37434; and
  - Apache HTTP Server 2.4.51 incorrectly associated with CVE-2021-42013.
- validation\_results.csvCSV

The importance of these failures is that the RAG system **knew about the vulnerability but did not correctly reason about the installed version**.

This directly supports the central design principle of the prototype: vulnerability detection requires both **CVE knowledge** and **version-specific applicability reasoning**.

## 7.8 Absence of JSON Schema Failures

No persistent JSON schema failures were observed in the 30-case validation. All three configurations achieved a **100% JSON validity rate**.

This result is significant because the system deliberately treats structured output as a separate engineering problem. The LLM engine uses JSON mode, parses the response, validates it against the Pydantic schema and retries when validation fails. `llm_engine.pyPY`

Therefore, the experiment demonstrates that the structured-output layer was reliable under the tested conditions.

However, the result should not be interpreted as proof that malformed output can never occur. It establishes only that no unresolved schema failure occurred within this particular 30-case validation.

## 7.9 Failure Analysis Conclusions

The failure analysis reveals three distinct layers of difficulty:

1. **Knowledge availability** — the model may not know the relevant vulnerability.
2. **Retrieval relevance** — the retrieval system may provide a related but incorrect vulnerability.
3. **Version reasoning** — the model may retrieve the correct vulnerability but incorrectly classify a patched version.

The full prototype addressed all three sufficiently for the tested 30 cases.

## 7.10 Limitations

Several limitations should be considered when interpreting the validation results:-

1. The validation dataset contains only 30 cases. It is therefore a controlled validation rather than a statistically large production benchmark.
2. The validation used a deliberately constructed set of vulnerable, patched and mismatch cases. Real enterprise environments contain substantially greater package diversity, versioning complexity and configuration variation.
3. The rag condition used a deterministic local retrieval mechanism for reproducible benchmarking rather than directly measuring every possible production embedding configuration.
4. The current validation did not produce observed examples of malformed json surviving the retry mechanism or hallucinated remediation advice. These categories therefore remain limitations to monitor rather than measured failure classes in this experiment.

## Chapter 8: Conclusion & Future Work

### 8.1 Conclusion

This research investigated whether open-source LLMs deployed entirely on-premise can provide useful AI-assisted vulnerability assessment capabilities in environments where cloud-hosted frontier models cannot be used because of data-residency, operational-security and infrastructure-sovereignty requirements.

The research proceeded in three stages. First, six open-source LLMs were comparatively evaluated across security-assessment dimensions including CVE accuracy, version reasoning, configuration analysis, structured output and handling of novel CVEs. Second, the comparative results were used to identify a suitable model configuration. Third, a working prototype was developed that integrates host artifact collection, local CVE enrichment, retrieval, local LLM inference, structured validation and reporting.

The results demonstrate that the problem cannot be solved effectively by using a language model in isolation. The comparative study showed substantial differences between candidate models, while the prototype validation demonstrated that retrieval and evidence-grounded reasoning are critical for reliable vulnerability classification.

The 30-case validation produced the strongest results for the full prototype configuration:

- **100% precision**
- **100% recall**
- **100% F1**
- **100% JSON validity**
- **100% CVSS-tier accuracy**

By comparison, the Base configuration achieved zero precision, recall and F1, while RAG achieved 45.45% precision, 90.91% recall and 60.61% F1.

The most important finding is therefore not simply that an LLM can identify vulnerabilities. Rather, the evidence demonstrates that **an on-premise vulnerability-assessment system benefits from combining deterministic evidence collection, local vulnerability retrieval, explicit version reasoning constraints and structured output validation.**

---

### 8.2 Answers to the Research Questions

**RQ1: What open-source LLMs are viable candidates for on-premise deployment?**

The comparative results identified **Qwen2.5:7B with mandatory RAG augmentation** as the preferred overall configuration. However, model selection is not determined solely by accuracy. Licence restrictions, latency, hardware requirements, structured-output reliability and deployment constraints must also be considered.

## **RQ2: How do candidate models compare across the security-assessment dimensions?**

The six-model study demonstrated substantial variation between models.

Some models performed strongly on known CVEs but poorly on novel vulnerabilities. Others showed stronger configuration or structured-output behaviour. In particular, the study demonstrated that RAG can substantially improve the performance of models on post-training-cutoff or novel CVEs.

The study therefore supports the conclusion that **there is no single universally optimal open-source model independent of architecture and task.**

For vulnerability assessment, the model must be evaluated together with its retrieval mechanism, prompting strategy, output constraints and deployment characteristics.

## **RQ3: Can a functional vulnerability-assessment prototype be built while keeping assessment data inside the operator's network boundary?**

The research demonstrates that this is technically feasible.

The implemented prototype collects host evidence locally, performs local CVE enrichment, uses local retrieval, performs inference through locally deployed Ollama models and validates the resulting JSON locally. The assessment CLI records `network_used: false`, and the assessment pipeline does not require cloud-based inference. `run_assessment.pyPY`

The prototype therefore demonstrates the feasibility of an AI-assisted vulnerability-assessment architecture in which sensitive scan artefacts remain inside the operator's environment.

## **8.3 Contributions of the Research**

The research makes three principal contributions.

### **1. Comparative evaluation**

A reproducible comparative evaluation of six open-source LLMs was performed using a 100-item security benchmark covering multiple vulnerability and configuration-analysis tasks. The study provides evidence about the strengths and limitations of locally deployable models rather than relying solely on general-purpose model benchmarks.

### **2. Evidence-grounded vulnerability-assessment architecture**

The project developed a modular architecture combining:

**local collection → CVE enrichment → retrieval → LLM reasoning → schema validation → reporting**

The architecture explicitly separates evidence collection from model reasoning and prevents uncertain candidate CVE matches from automatically becoming confirmed vulnerabilities.

### **3. On-premise prototype**

A working prototype was implemented using local host collection, local vulnerability data, local retrieval, local LLM inference and structured reporting. The architecture provides a practical foundation for future integration into security operations workflows while maintaining the requirement that assessment data remain inside the operator's network boundary.

## **8.4 Future Work**

### **8.4.1 Fine-Tuning on Security-Specific Data**

The next stage of the research could investigate parameter-efficient fine-tuning using a security-specific instruction dataset.

A dataset containing approximately 100–200 carefully verified CVE instruction pairs could be used to teach the model patterns such as:

- affected-version interpretation;
- fixed-version reasoning;
- CVE/product disambiguation;
- severity classification;
- evidence-grounded remediation; and
- explicit uncertainty handling.

QLoRA or a similar parameter-efficient approach could be evaluated against the current RAG-only configuration.

The key research question would be whether fine-tuning improves version reasoning without increasing hallucination or false-positive rates.

### **8.4.2 Security-Specific Cross-Encoder Ranking**

The current retrieval architecture can be extended with a security-specific cross-encoder reranker.

Rather than relying only on semantic similarity, candidate CVEs could be ranked using features such as:

- package identity;
- vendor;
- product;
- version range;
- CVE description;
- CPE information; and
- distribution-specific advisory information.

#### **8.4.3 Expanded Windows Server Coverage**

The current implementation demonstrates Windows artifact collection, but production deployment would require deeper Windows Server coverage.

Future collectors should expand coverage of:

- Windows Server 2019 and 2022;
- installed Windows features;
- scheduled tasks;
- local users and groups;
- firewall configuration;
- registry security settings;
- service permissions;
- PowerShell configuration;
- IIS configuration; and
- Windows-specific security policies.

This would make the prototype more representative of real TSP infrastructure rather than a limited host-inventory environment.

#### **8.4.4 Larger Validation Dataset**

The 30-case validation provides useful evidence but should be extended to a substantially larger dataset. A future benchmark should include:

- hundreds of package/version cases;
- multiple operating systems;
- vendor-specific package naming;
- vulnerable and patched versions;
- version-range edge cases;
- configuration vulnerabilities;
- container vulnerabilities;
- Kubernetes misconfigurations; and
- multi-artifact exploitability scenarios.

- A larger dataset would allow confidence intervals and stronger statistical comparison between model configurations.

#### 8.4.5 SIEM Integration

The structured JSON architecture provides a natural interface for future SIEM integration. The prototype could emit findings into systems such as a security-information and event-management platform using fields such as:

- host;
- component;
- installed version;
- CVE;
- severity;
- confidence;
- evidence;
- remediation; and
- timestamp.

This would allow AI-assisted vulnerability findings to participate in existing security monitoring, ticketing and incident-response workflows. The structured schema is particularly suitable for this purpose because the LLM output is validated before it enters downstream processing.

#### 8.4.6 Continuous Local CVE Updates

The prototype currently separates CVE data acquisition from assessment. Future deployments could introduce a controlled update mechanism in which a trusted update process periodically retrieves and verifies new vulnerability information before storing it locally. The assessment environment would continue to operate exclusively against the local cache.

This would preserve the data-residency properties of the scanner while reducing the knowledge gap associated with newly disclosed vulnerabilities.

### 8.5 Final Statement

The central conclusion of this research is that **on-premise open-source LLMs are technically capable of supporting vulnerability assessment when they are deployed as part of an evidence-grounded security architecture rather than used as standalone vulnerability databases.**

The comparative study established the importance of model selection and retrieval augmentation, while the prototype validation demonstrated the additional importance of exact evidence matching, version reasoning and structured validation.

The resulting architecture provides a practical foundation for privacy-preserving AI-assisted vulnerability assessment in environments where sensitive infrastructure information cannot be sent to external cloud LLM services. Its current limitations provide clear directions for further research and production hardening.

# REFERENCES

- [1]Ministry of Electronics and Information Technology, *THE DIGITAL PERSONAL DATA PROTECTION ACT, 2023 (NO. 22 OF 2023)*. Accessed 5 Aug 2026[online]. Available:<https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf>
- [2]Ministry of Electronics and Information Technology, *NCIIPC-Rules-notification*. Accessed 5 Aug 2026[online]. Available: <https://www.meity.gov.in/static/uploads/2024/02/NCIIPC-Rules-notification-1.pdf>
- [3]India Code, *THE INFORMATION TECHNOLOGY ACT, 2000* \_\_, *Chapter IX, Section 70B*. Accessed 5 Aug 2026[online]. Available: [https://www.indiacode.nic.in/bitstream/123456789/13116/1/it\\_act\\_2000\\_updated.pdf](https://www.indiacode.nic.in/bitstream/123456789/13116/1/it_act_2000_updated.pdf)
- [4]J. Wei *et al.*, ‘Chain-of-Thought Prompting Elicits Reasoning in Large Language Models’, Jan. 10, 2023, *arXiv*: arXiv:2201.11903. doi: 10.48550/arXiv.2201.11903.
- [5]A. Q. Jiang *et al.*, ‘Mistral 7B’, Oct. 10, 2023, *arXiv*: arXiv:2310.06825. doi: 10.48550/arXiv.2310.06825.
- [6]M. Abdin *et al.*, ‘Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone’, Aug. 30, 2024, *arXiv*: arXiv:2404.14219. doi: 10.48550/arXiv.2404.14219.
- [7]G. Team *et al.*, ‘Gemma 2: Improving Open Language Models at a Practical Size’, Oct. 02, 2024, *arXiv*: arXiv:2408.00118. doi: 10.48550/arXiv.2408.00118.
- [8]DeepSeek-AI *et al.*, ‘DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence’, Jun. 17, 2024, *arXiv*: arXiv:2406.11931. doi: 10.48550/arXiv.2406.11931.
- [9]Qwen *et al.*, ‘Qwen2.5 Technical Report’, Jan. 03, 2025, *arXiv*: arXiv:2412.15115. doi: 10.48550/arXiv.2412.15115.
- [10]Z. Yang *et al.*, ‘Llama-3.1-FoundationAI-SecurityLLM-Reasoning-8B Technical Report’, Jan. 28, 2026, *arXiv*: arXiv:2601.21051. doi: 10.48550/arXiv.2601.21051.
- [11]‘NVD - Home’. Accessed: Sep. 08, 2026. [Online]. Available: <https://nvd.nist.gov/vuln/detail/cve-2024-21626#vulnCurrentDescriptionTitle>
- [12]‘CVE-2023-4911 Looney Tunables Exploit: Linux GLIBC Vulnerability’. Accessed: Sep. 08, 2026. [Online]. Available: <https://www.uptycs.com/blog/threat-research-report-team/cve-2023-4911-looney-tunables-glibc-exploit>
- [13]P. Lewis *et al.*, ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’, Apr. 12, 2021, *arXiv*: arXiv:2005.11401. doi: 10.48550/arXiv.2005.11401.
- [14]‘Getting Started’, Chroma Docs. Accessed: Sep. 08, 2026. [Online]. Available: <https://docs.trychroma.com/docs/overview/getting-started>



- [15] 'IEEE Reference Style Guide for Authors'. Accessed: Sep. 08, 2026. [Online]. Available: [https://docs.google.com/document/d/e/2PACX-1vSBoV\\_hTIURPZKtQkxI6bAONJtjyDLxQIUvCHi-XPzD8NKLcxRfZrpvQLFrjWZ3M6FFR\\_Obau3R-t63/pub](https://docs.google.com/document/d/e/2PACX-1vSBoV_hTIURPZKtQkxI6bAONJtjyDLxQIUvCHi-XPzD8NKLcxRfZrpvQLFrjWZ3M6FFR_Obau3R-t63/pub)
- [16] V. Mohiley, *Vanshika-mohiley/jio\_internship*. (Sep. 07, 2026). Python. Accessed: Sep. 08, 2026. [Online]. Available: [https://github.com/Vanshika-mohiley/jio\\_internship](https://github.com/Vanshika-mohiley/jio_internship)
- [17] Vanshika-mohiley, 'jio\_internship/100\_itemdataset at main · Vanshika-mohiley/jio\_internship', GitHub. Accessed: Sep. 08, 2026. [Online]. Available: [https://github.com/Vanshika-mohiley/jio\\_internship/tree/main/100\\_itemdataset](https://github.com/Vanshika-mohiley/jio_internship/tree/main/100_itemdataset)
- [18] 'ZoteroBib: Fast, free bibliography generator - MLA, APA, Chicago, Harvard citations'. Accessed: Sep. 08, 2026. [Online]. Available: <https://zbib.org/>
- [19] 'Introduction', Ollama. Accessed: Sep. 10, 2026. [Online]. Available: <https://docs.ollama.com/api/introduction>
- [20] 'Holistic Evaluation of Language Models (HELM)'. Accessed: Sep. 10, 2026. [Online]. Available: <https://crfm.stanford.edu/helm/long-context/latest/>
- [21] *CIS\_Kubernetes\_Benchmark\_V2.0.1*, Center for Internet Security, CIS Benchmarks, v2.0.1 - 04-29-2026, pp. 7 – 291.
- [22] *CIS\_Red\_Hat\_Enterprise\_Linux\_9\_Benchmark\_v3.0.0*, Center for Internet Security, CIS Benchmarks, v3.0.0 - 08-28-2026, pp. 5 – 1079.
- [23] 'Pydantic Validation', Pydantic Docs. Accessed: Sep. 10, 2026. [Online]. Available: <https://pydantic.dev/docs/validation/latest/get-started/>
- [24] 'Streamlit • A faster way to build and share data apps'. Accessed: Sep. 10, 2026. [Online]. Available: <https://streamlit.io/>
- [25] 'unittest — Unit testing framework', Python documentation. Accessed: Sep. 10, 2026. [Online]. Available: <https://docs.python.org/3/library/unittest.html>